



Computer Vision p2

2-08-2026

Hebah Omer Soleman

Attendance



Image Segmentation

Table of Contents

1. Introduction
2. Image Segmentation
3. Adapting CNNs to Segmentation Tasks
4. Upsampling Operations
5. Residual Connections and U-Net
6. Instance and Panoptic Segmentation

Image Classification

- Previously, we discussed Image Classification
- A core task in Computer Vision



This image by Nikita is
licensed under CC-BY 2.0

(assume given a set of possible labels)
{dog, cat, truck, plane, ...}



cat

Computer Vision Tasks

Classification



CAT

No spatial extent

Semantic Segmentation



**GRASS, CAT,
TREE, SKY**

No objects, just pixels

Object Detection



DOG, DOG, CAT

Multiple Object

Instance Segmentation



DOG, DOG, CAT

[This image is CC0 public domain](#)

Semantic Segmentation



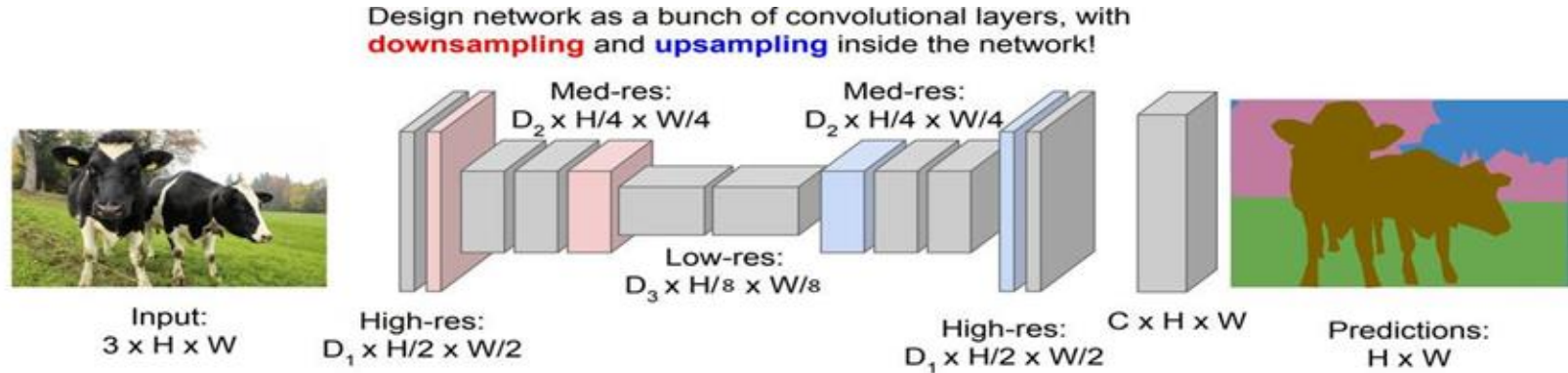
GRASS, CAT,
TREE, SKY, ...

Paired training data: for each training image, each pixel is labeled with a semantic category.



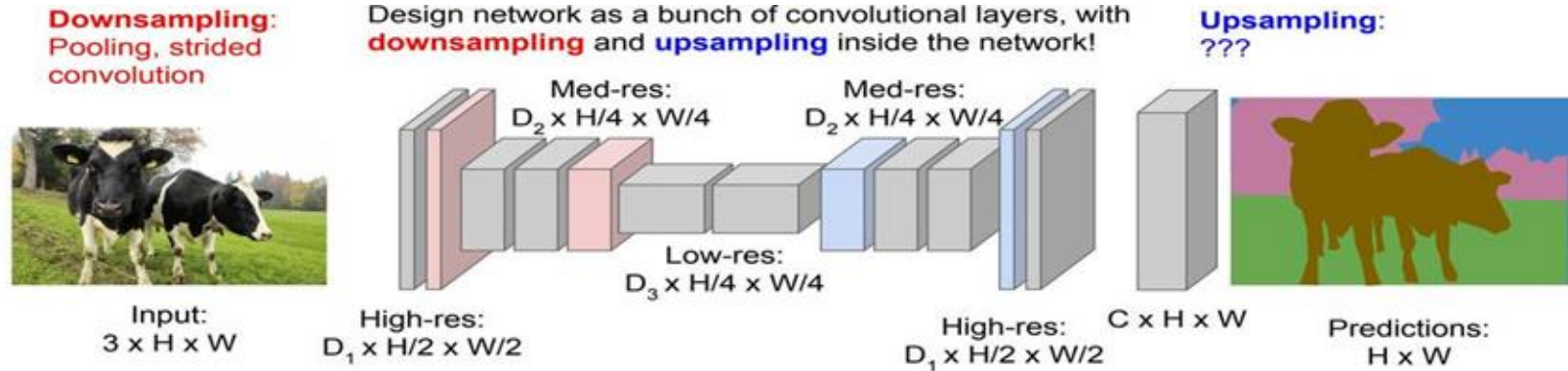
At test time, classify each pixel of a new image.

Semantic Segmentation Idea: Fully Convolutional (cont.)



Long, Shelhamer, and Darrell, "Fully Convolutional Networks for Semantic Segmentation", CVPR 2015
Noh et al., "Learning Deconvolution Network for Semantic Segmentation", ICCV 2015

Semantic Segmentation Idea: Fully Convolutional (cont.)



Long, Shelhamer, and Darrell, "Fully Convolutional Networks for Semantic Segmentation", CVPR 2015
Noh et al., "Learning Deconvolution Network for Semantic Segmentation", ICCV 2015

In-Network Upsampling: Unpooling

Nearest Neighbor

1	2
3	4



1	1	2	2
1	1	2	2
3	3	4	4
3	3	4	4

Input: 2 x 2

Output: 4 x 4

“Bed of Nails”

1	2
3	4

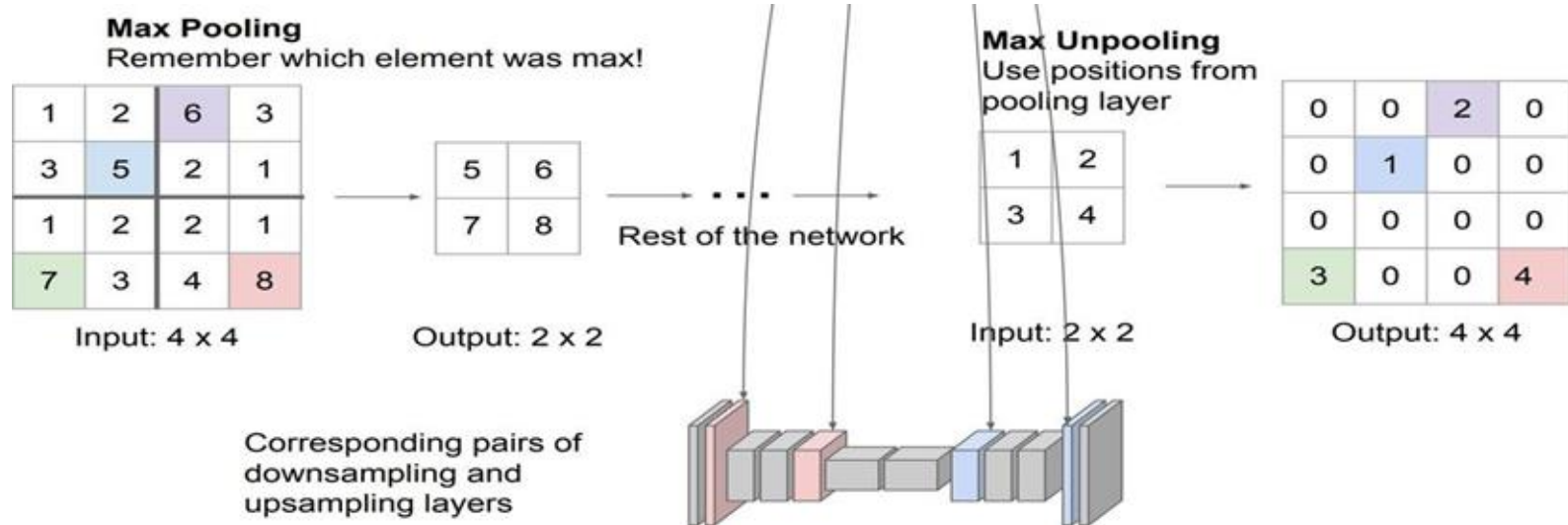


1	0	2	0
0	0	0	0
3	0	4	0
0	0	0	0

Input: 2 x 2

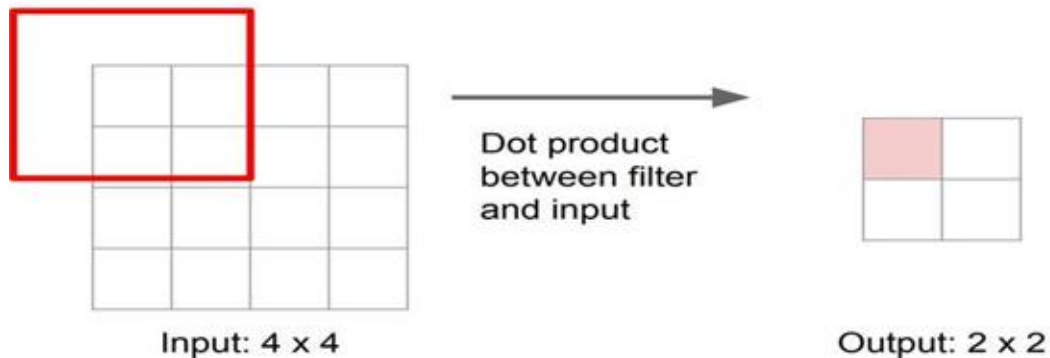
Output: 4 x 4

In-Network Upsampling: Max Unpooling



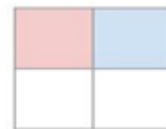
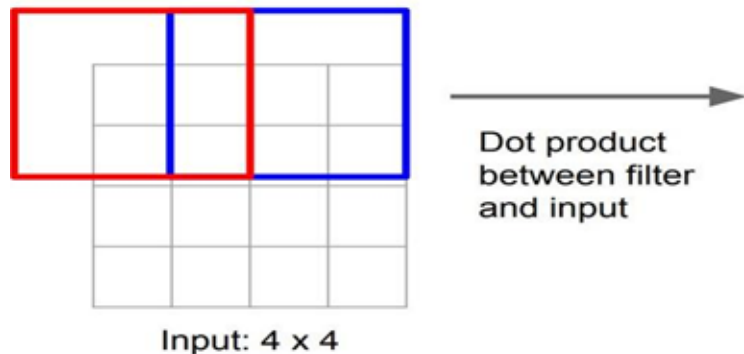
Learnable Upsampling: Transposed Convolution

Recall: Normal 3 x 3 convolution, stride 2 pad 1



Learnable Upsampling: Transposed Convolution (cont.)

Recall: Normal 3 x 3 convolution, stride 2 pad 1



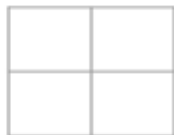
Filter moves 2 pixels in the input for every one pixel in the output

Stride gives ratio between movement in input and output

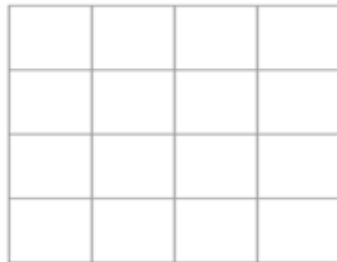
We can interpret strided convolution as "learnable downsampling".

Learnable Upsampling: Transposed Convolution (cont.)

3 x 3 **transposed** convolution, stride 2 pad 1



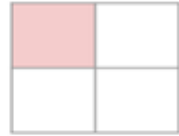
Input: 2 x 2



Output: 4 x 4

Learnable Upsampling: Transposed Convolution (cont.)

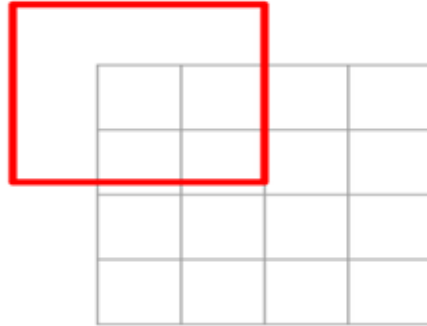
3 x 3 **transposed** convolution, stride 2 pad 1



Input: 2 x 2



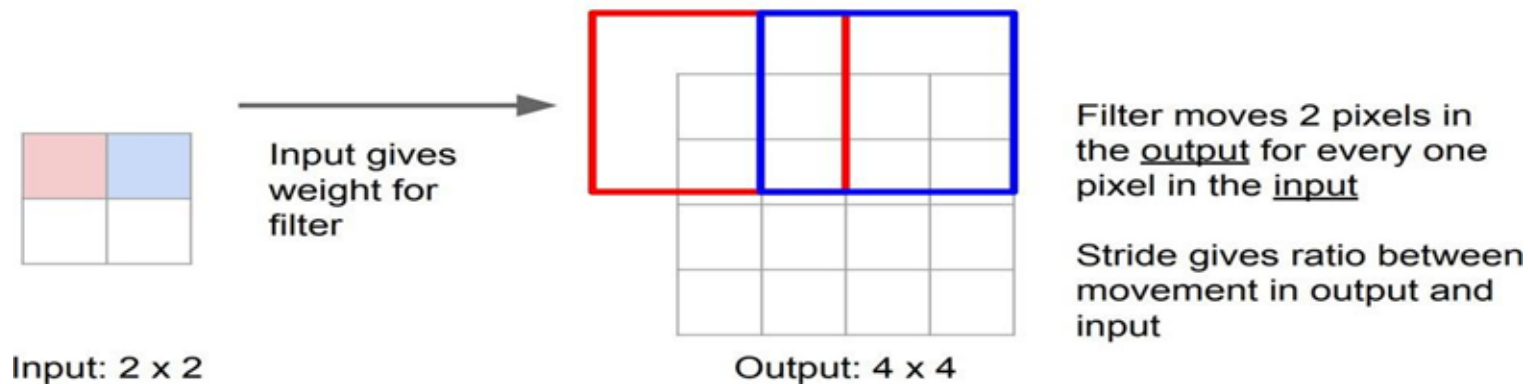
Input gives
weight for
filter



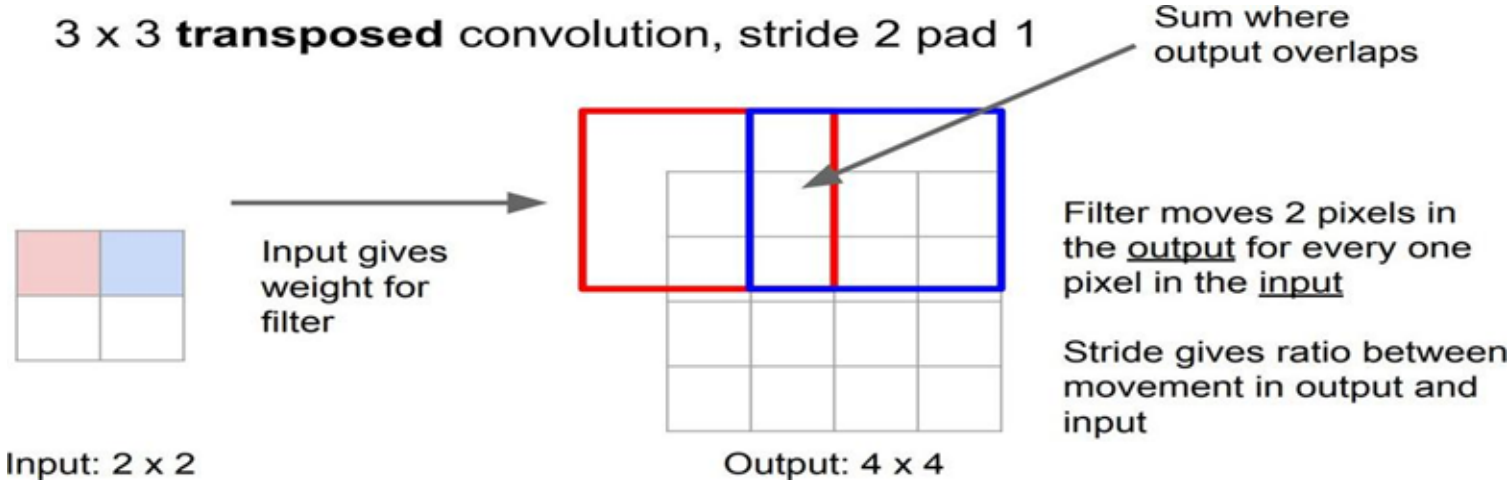
Output: 4 x 4

Learnable Upsampling: Transposed Convolution (cont.)

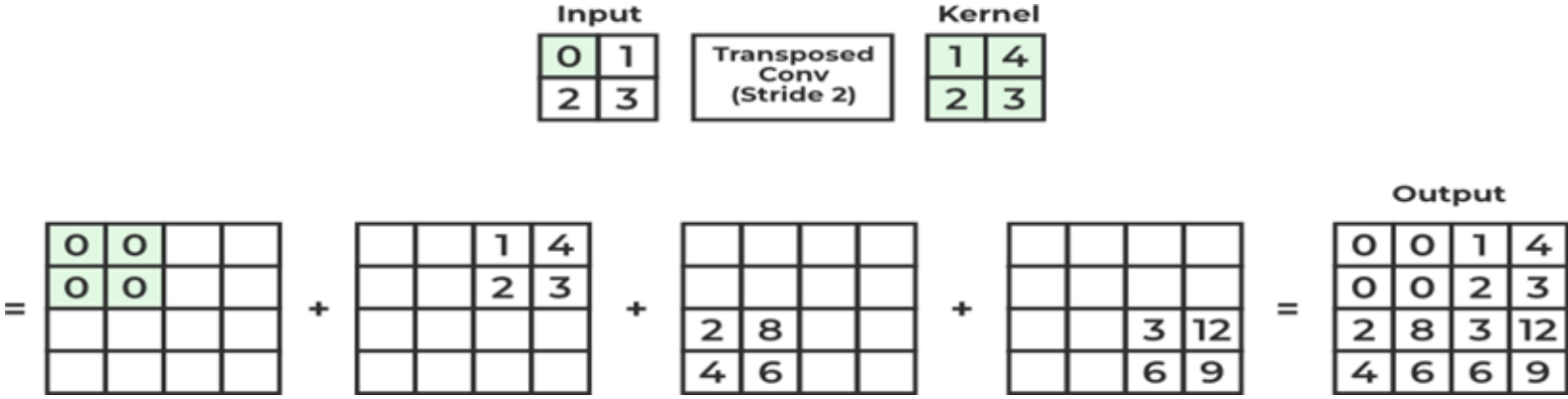
3 x 3 **transposed** convolution, stride 2 pad 1



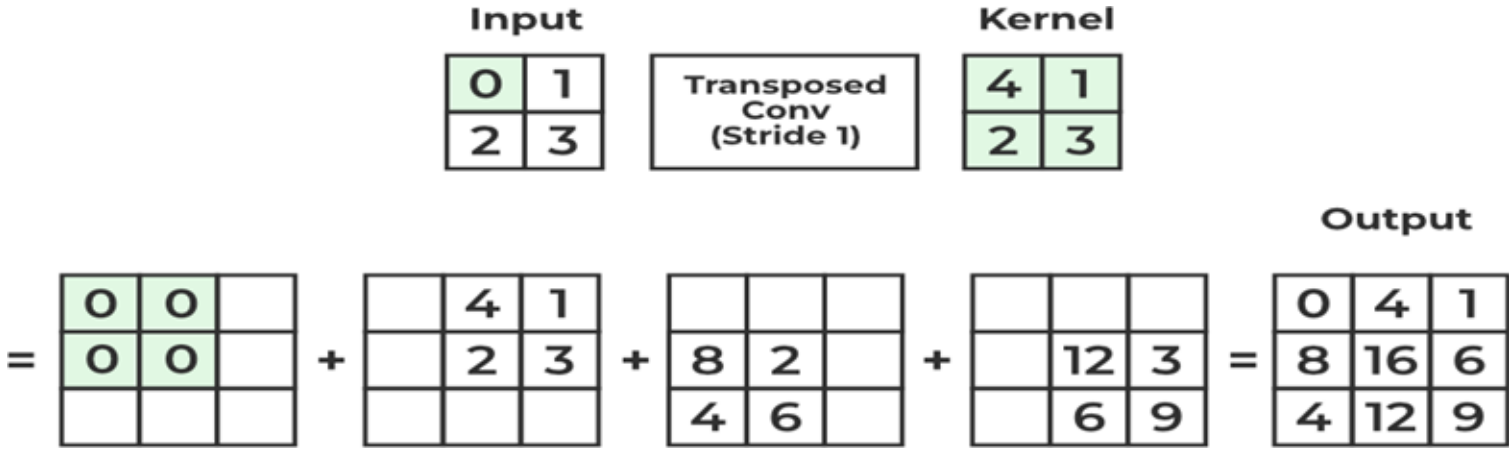
Learnable Upsampling: Transposed Convolution (cont.)



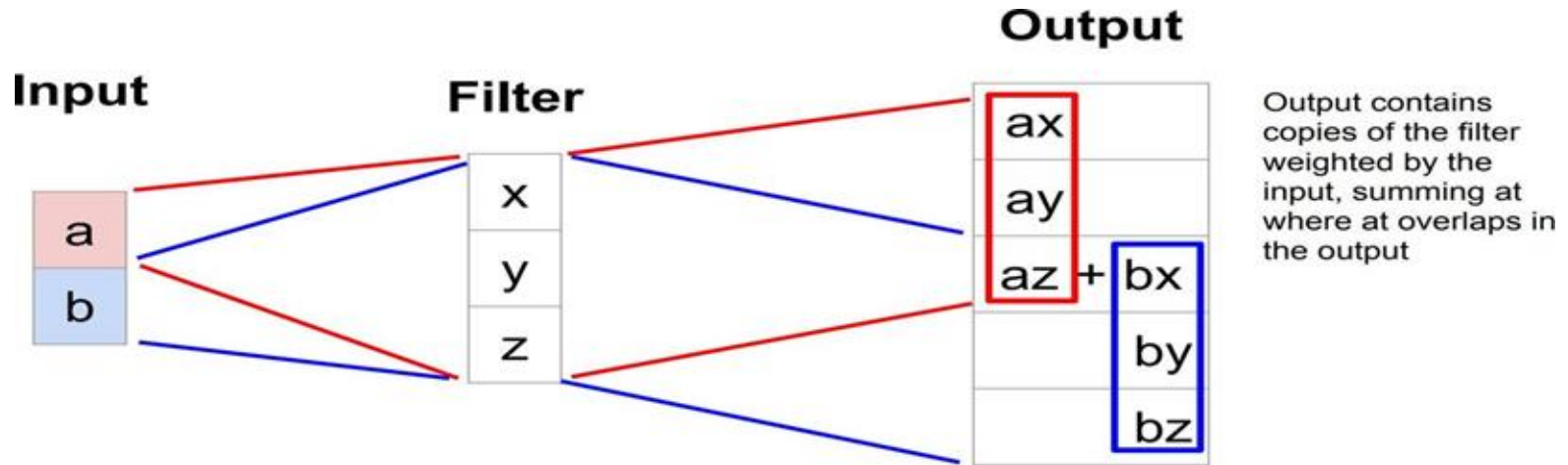
Learnable Upsampling: Transposed Convolution (cont.)



Learnable Upsampling: Transposed Convolution (cont.)



Transposed Convolution: 1D Example



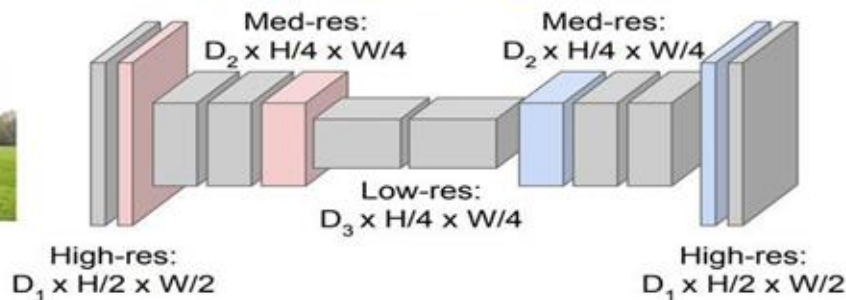
Semantic Segmentation Idea: Fully Convolutional

Downsampling:
Pooling, strided
convolution



Input:
 $3 \times H \times W$

Design network as a bunch of convolutional layers, with **downsampling** and **upsampling** inside the network!



Upsampling:
Unpooling or strided
transposed convolution



Predictions:
 $H \times W$

Can We Do Better?

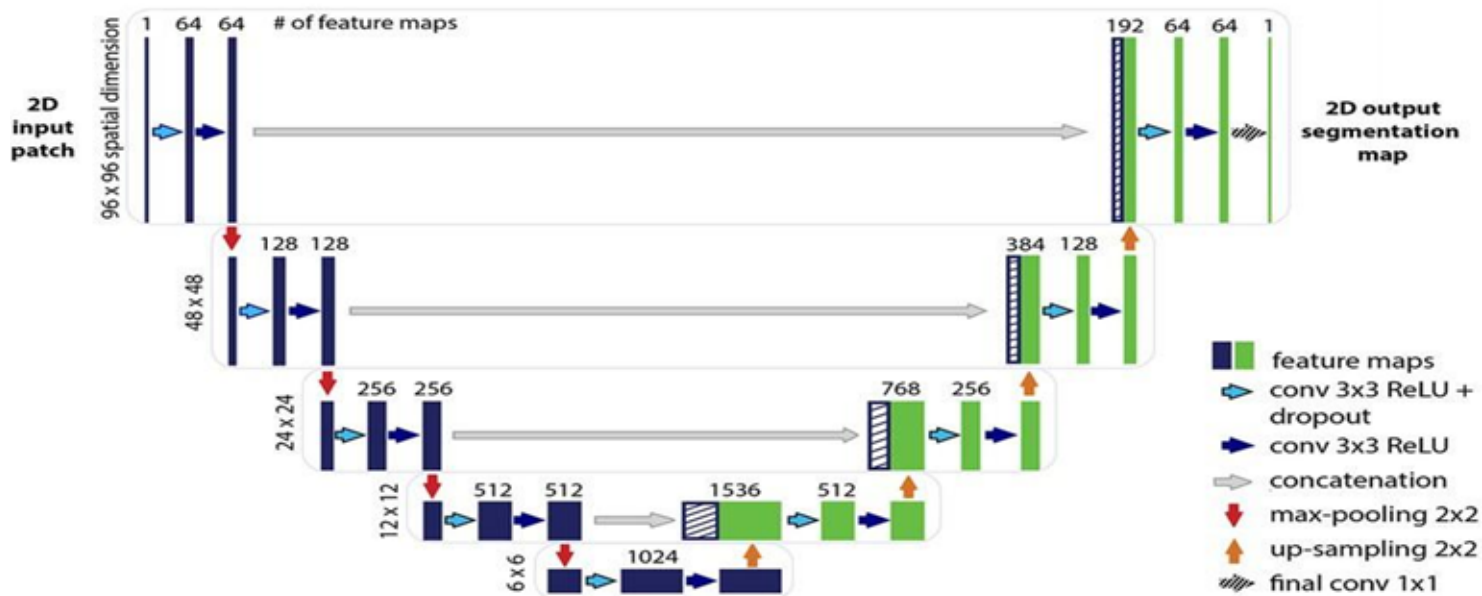
- The downsampling-then-upsampling approach works well for semantic segmentation.
- **But... can we do better?**
- **Problem** tsol eb yam noitamrofni laitaps dna sliated tnatropmI :
.gnilpmasnwod gnirud

Can We Do Better?

- The downsampling-then-upsampling approach works well for semantic segmentation.
- **But... can we do better?**
- **Problem** tsol eb yam noitamrofni laitaps dna sliated tnatropmI : during downsampling.
- **Solution** ecudortnI : **residual connections** to preserve spatial information.

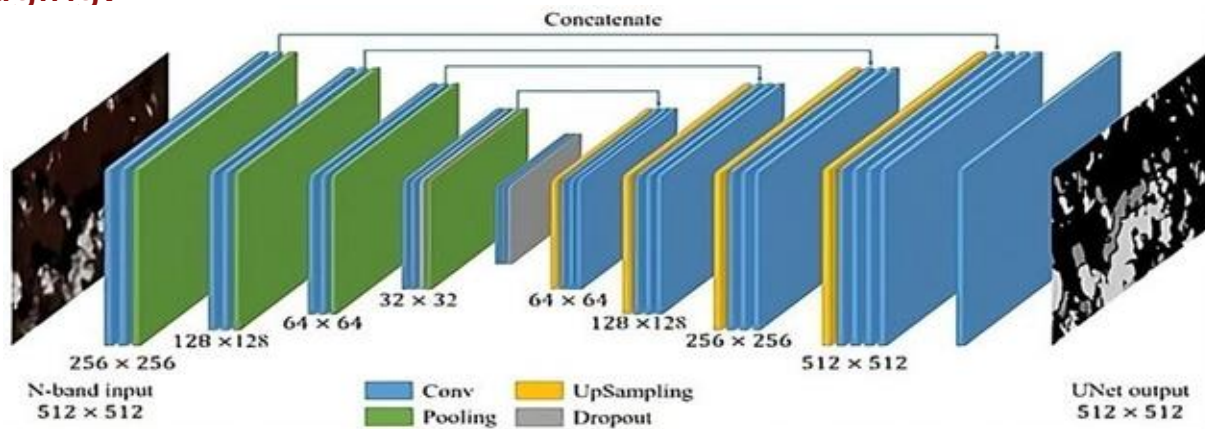
Residual Connections in Segmentation

- Directly connect features from downsampling layers to upsampling layers.
- Help recover lost spatial details and improve segmentation accuracy.



U-Net

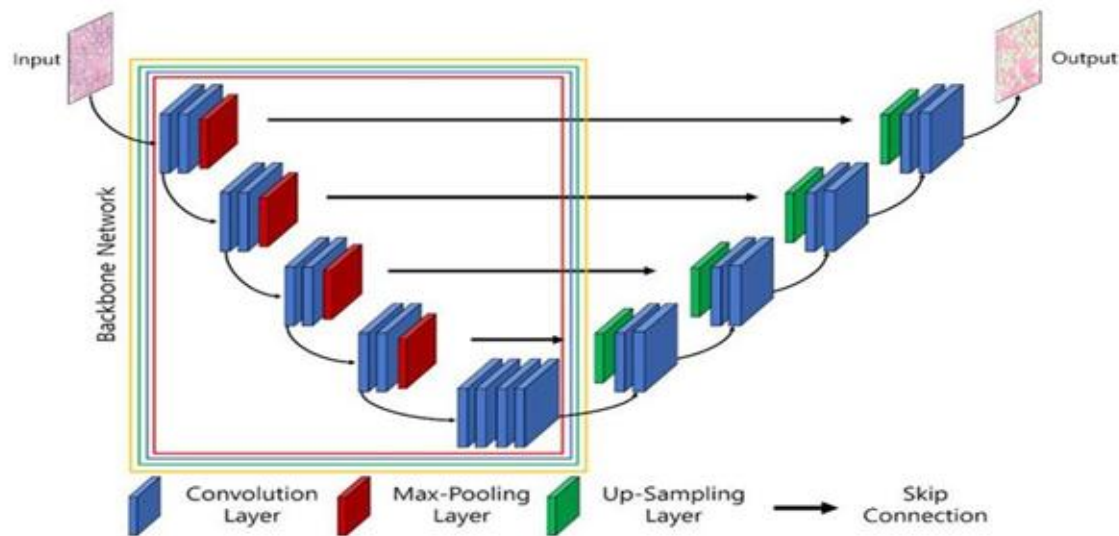
- This architecture, with residual connections, is called **U-Net**.
- **Why the name?**
 - The architecture resembles the shape of the letter "U."
 - Features are downsampled in the encoder and upsampled in the decoder, with skip connections in between.
- U-Net is widely used for segmentation tasks, especially in biomedical imaging.



Using a Pretrained U-Net

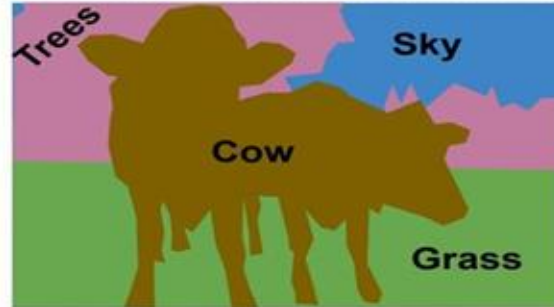
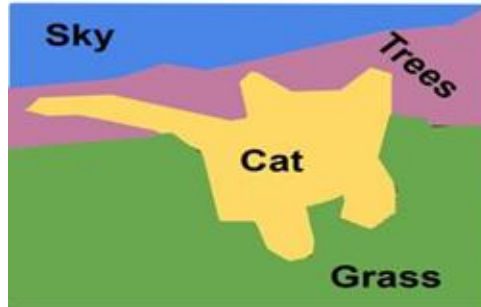
- **Pretrained Encoder:**

- The encoder can use a pretrained backbone (e.g., ResNet, EfficientNet).
- This help utilize features learned on large datasets (e.g., ImageNet).
- Only the decoder is trained from scratch for segmentation-specific tasks.



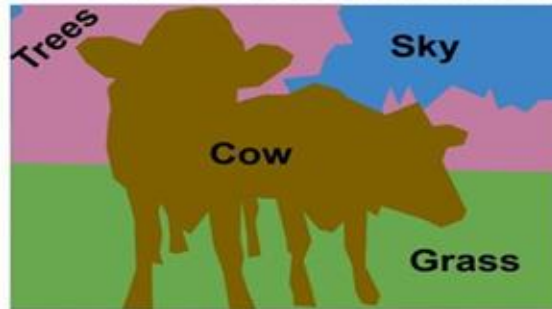
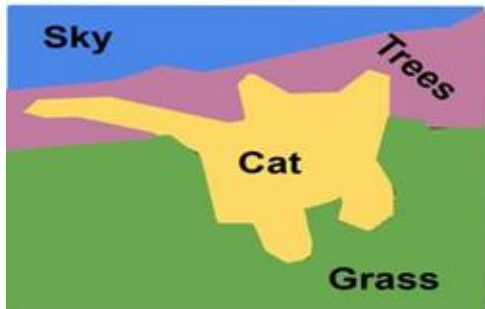
Semantic Segmentation

- Label each pixel in the image with a category label



Semantic Segmentation

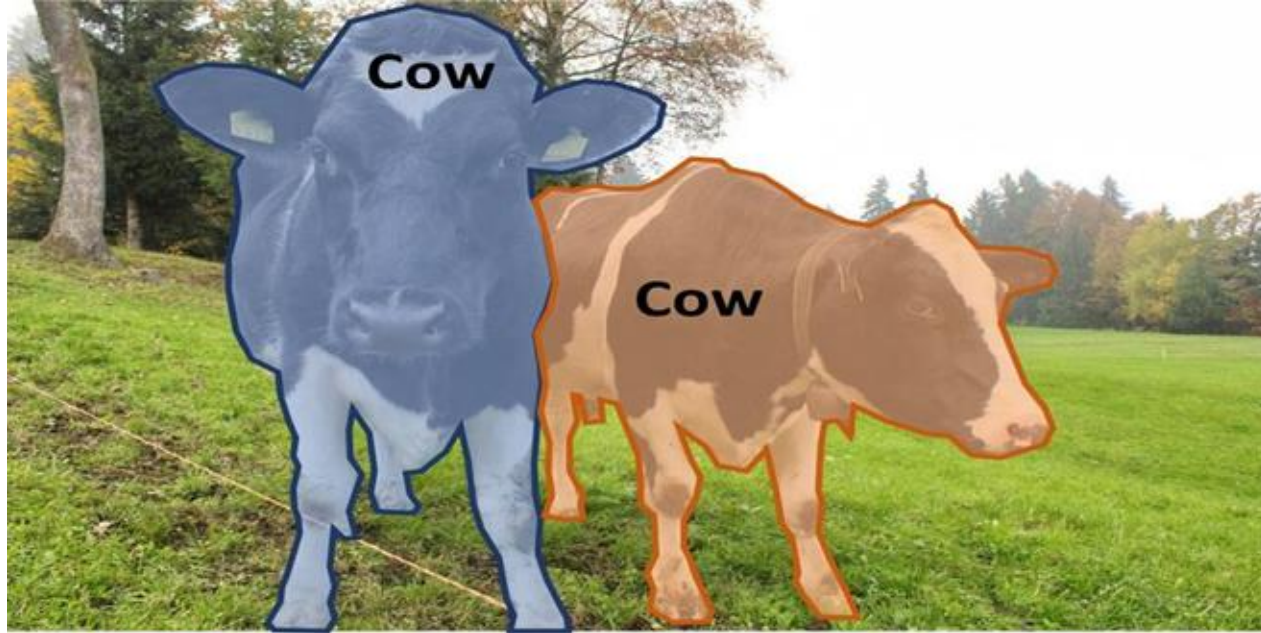
- Label each pixel in the image with a category label



- Does not differentiate instances, only care about pixels

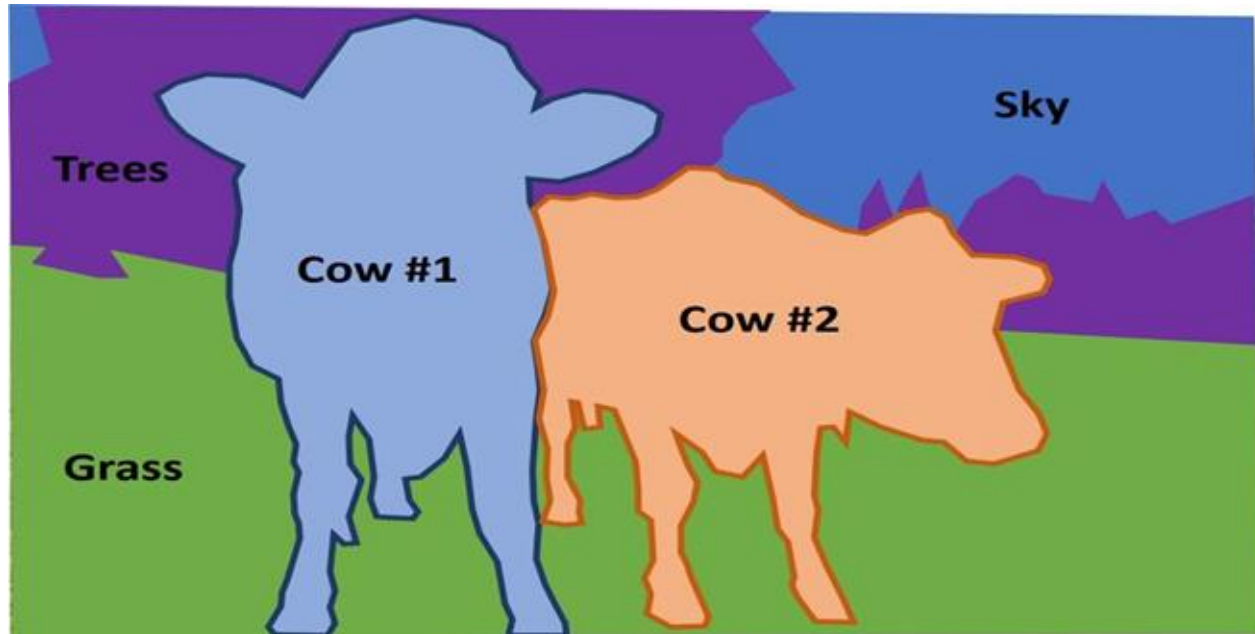
Instance Segmentation

- Separate object instances, but only things



Panoptic Segmentation

- Label all pixels in the image (both things and stuff)



References

These slides have been adapted from

- Fei-Fei Li, Yunzhu Li & Ruohan Gao, Stanford CS231n: [Deep Learning for Computer Vision](#)
- Assaf Shocher, Shai Bagon, Meirav Galun & Tali Dekel, WAIC DL4CV [Deep Learning for Computer Vision: Fundamentals and Applications](#)
- Justin Johnson, UMich EECS [rof gninraeL peeD :498.008/598.008 Computer Vision](#)